

Skill 3

1. To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Manage Linked Lists, Release Memory Correctly, Verify with Valgrind

Output:

```
kannamina_abhishek@LAPTOP-D9F9IT80:~$ gcc -wall -wextra history_memory.c -o history_memory
gcc: error: unrecognized command-line option '-wall'; did you mean '-Wall'?
gcc: error: unrecognized command-line option '-wextra'; did you mean '-Wextra'?
kannamina_abhishek@LAPTOP-D9F9IT80:~$ gcc -Wall -Wextra history_memory.c -o history_memory
kannamina_abhishek@LAPTOP-D9F9IT80:~$ ./history_memory
Simple Shell - History & Dynamic Memory
=====
Type commands. Use:
  history - show command history
  exit    - quit
  \033[A  - previous command (concept)
  \033[B  - next command (concept)

shell> valgrind --leak-check=full ./history_memory
Command stored: valgrind --leak-check=full ./history_memory

shell> |
```

Question:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 10

// Linked list node for command history
typedef struct HistoryNode {
    char *command;
    struct HistoryNode *next;
} HistoryNode;

// Add command to history
void addHistory(HistoryNode **head, const char *command) {
    HistoryNode *newNode = malloc(sizeof(HistoryNode));

    if (newNode == NULL) {
        perror("Memory allocation failed");
        exit(1);
    }

    newNode->command = malloc(strlen(command) + 1);

    if (newNode->command == NULL) {
        perror("Memory allocation failed");
        free(newNode);
        exit(1);
    }

    strcpy(newNode->command, command);
    newNode->next = NULL;

    if (*head == NULL) {
        *head = newNode;
    } else {
        HistoryNode *temp = *head;

        while (temp->next != NULL)
            temp = temp->next;

        temp->next = newNode;
    }
}

// Display command history
void displayHistory(HistoryNode *head) {
    int number = 1;

    printf("\n--- Command History ---\n");

    while (head != NULL) {
        printf("%d: %s\n", number, head->command);
        head = head->next;
        number++;
    }
}

// Free linked list memory
void freeHistory(HistoryNode *head) {
    HistoryNode *temp;

    while (head != NULL) {
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^/_ Go To Line	M-E Redo

```

        bufferSize *= 2;

        char *temp = realloc(buffer, bufferSize);

        if (temp == NULL) {
            perror("Buffer resizing failed");
            free(buffer);
            freeHistory(history);
            return 1;
        }

        buffer = temp;

        if (fgets(buffer + currentLength,
            bufferSize - currentLength,
            stdin) == NULL)
            break;
    }

    // Remove newline
    buffer[strcspn(buffer, "\n")] = '\0';

    // Empty input
    if (strlen(buffer) == 0)
        continue;

    // Exit
    if (strcmp(buffer, "exit") == 0)
        break;

    // Display history
    if (strcmp(buffer, "history") == 0) {
        displayHistory(history);
        continue;
    }

    // Escape sequence detection
    if (strcmp(buffer, "\033[A") == 0) {
        printf("Previous command requested.\n");
        continue;
    }

    if (strcmp(buffer, "\033[B") == 0) {
        printf("Next command requested.\n");
        continue;
    }

    // Store command in history
    addHistory(&history, buffer);

    printf("Command stored: %s\n", buffer);
}

// Release memory
free(buffer);
freeHistory(history);

printf("\nAll allocated memory released.\n");

return 0;
}
|

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]F Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify

[^]C Location
[^]/ Go To Line

^M-^U Undo
^M-^E Redo